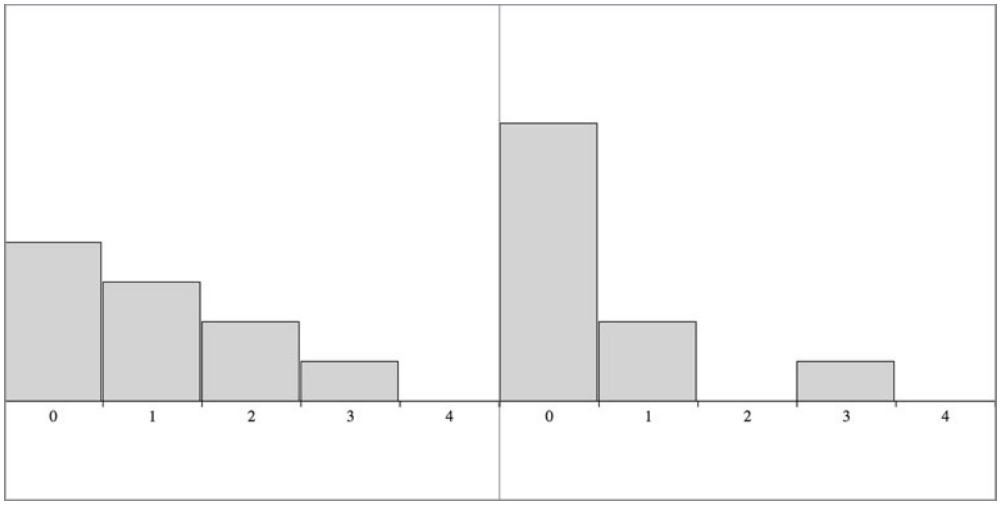


Figure 5.2 The histogram in its initial state (left) and after we change the measure from favorites to retweets (right) by clicking on one of the bars.

You're not expected to follow the process of using the histogram to create the results in figure 5.2. You'll get into that as you look at more layouts throughout this chapter. Notice a few general principles: first, a layout formats the data for display, as I pointed out in the beginning of chapter 4. Second, you still need the same scales and components that you needed when you created a bar chart from raw data without the help of a layout. Third, the histogram is useful because it automatically bins data, whether it's whole numbers like this or it falls in a range of values in a scale. Finally, if you want to dynamically change a chart using a different dimension of your data, you don't need to remove the original. You just need to reformat your data using the layout and rebind it to the original elements, preferably with a transition. You'll see this in more detail in your next example, which uses another type of chart: pie charts.

5.2 Pie charts

One of the most straightforward layouts available in D3 is the pie layout, which is used to make pie charts like those shown in figure 5.3. Like all layouts, a pie layout can be created, assigned to a variable, and used as both an object and a function. In this section you'll learn how to create a pie chart and transform it into a ring chart. You'll also learn how to use tweening to properly transition it when you change its data source. After you create it, you can pass it an array of values (which I'll refer to as a dataset), and it will compute the necessary starting and ending angles for each of those values to draw a pie chart. When we pass an array of numbers as our dataset to a pie layout in the console as in the following code, it doesn't produce any kind of graphics but rather results in the response shown in figure 5.4:

```
var pieChart = d3.layout.pie();
var yourPie = pieChart([1,1,2]);
```